

KILIE IMMO — Architecture Backend

Lead Backend Developer — Framework Custom PHP (style Microsoft Dynamics NAV)

1. ANALYSE DU FRAMEWORK EXISTANT

Le projet repose sur un **framework PHP propriétaire** inspiré de **Microsoft Dynamics NAV / Business Central**. Ce choix architectural est central : toute extension doit respecter scrupuleusement ses conventions.

Composants du framework

Composant	Rôle	Équivalent NAV
Table	ORM custom — définit la structure, les clés, les triggers, les requêtes SQL	Table Object
Codeunit	Classe de logique métier — méthodes statiques uniquement	Codeunit Object
Enum	Énumération PHP 8.1 typée — valeurs de statut et listes fermées	Enum Object
Page	<i>(structure prévue, non utilisée côté API)</i>	Page Object
Routes.php	Registre centralisé de toutes les routes REST	—
App::route()	Déclarateur de route REST	—

Pattern ORM (Table)

```
// Instancier
$bail = new Bail();

// Filtrer
$bail->setRange('Tenant_code', $tenantCode);           // WHERE Tenant_code = ?
$bail->setFilter('Statut', '=%1|=%2', 'Actif', 'Signé'); // WHERE Statut IN (...)

// Lire
$bail->FindFirst();           // Premier enregistrement correspondant
$bail->FindSet();             // Tous les enregistrements correspondants → $bail->recordSet
$bail->get($codeKey, $tenantCode); // Accès par clé primaire
$bail->Count();               // Compter

// Écrire
$bail->Validate('Champ', $valeur); // Affecte + déclenche onValidate()
$bail->Insert();                   // INSERT + retourne ['status'=>201, ...]
$bail->Modify();                   // UPDATE + retourne ['status'=>200, ...]
$bail->Delete();                   // DELETE

// Triggers (à surcharger dans chaque Table)
public function onInsert() {}
public function onModify() {}
public function onDelete() {}
```

Pattern Codeunit

```
class BailManagement {
    public static function creerBail(array $data): string {
        header('Content-Type: application/json');
```

```
// 1. Auth + Tenant
$token = getAthorizationToken();
if (!AuthenticationManagement::auth($token)) Unauthorized('Session expirée');
$tenantCode = getSessionTenant($token);

// 2. Validation
$rules = ['No_Bail' => ['required' => true], 'Unite_code' => ['required' => true]];
$validation = Security::validateInput($data, $rules);
if ($validation !== true) return json_encode(['status' => 400, 'message' =>
end($validation)]);

// 3. Logique métier
$bail = new Bail();
foreach ($data as $k => $v) $bail->Validate($k, Security::sanitizeInput($v, 'string'));
$bail->Validate('Tenant_code', $tenantCode);
$result = $bail->Insert();

// 4. Événement (si EventDispatcher activé)
EventDispatcher::publish('BailCree', ['bail_no' => $bail->No->value]);

// 5. Commit + réponse
db->commit();
return json_encode($result);
}
}
```

Pattern Route

```
App::route('POST', '/baux', function () {
    $data = getSecureJsonInput();
    echo BailManagement::creerBail($data ?? []);
});

App::route('GET', '/baux/{no}', function (string $no) {
    echo BailManagement::getBail(urldecode($no));
});
```

2. CONVENTIONS DE CODE

2.1 Numérotation des objets (NAV Object ID)

Plage	Module	Rôle
9990000–9999999	Système (Base system apps)	Réservé framework — ne pas modifier
50000–50999	Paramètres & Abonnements	Config SaaS, plans, licences
51000–51999	Gestion immobilière	Biens, Unités, Mandats
52000–52999	Parties prenantes	Locataires, Propriétaires, KYC
53000–53999	Baux & Contrats	Baux, Échéanciers, EDL
54000–54999	Paiements & Finance	Paiements, MoMo, Quittances
55000–55999	Comptabilité	Plan comptable, Écritures
56000–56999	Maintenance	OI, Prestataires, Devis

Plage	Module	Rôle
57000–57999	CRM	Prospects, Activités
58000–58999	Ventes immobilières	Mandats vente, Offres, Compromis
59000–59999	GED	Documents, Dossiers
60000–60999	Notifications	Templates, File, Logs
61000–61999	Intelligence artificielle	Solvabilité, Estimation
62000–62999	Reporting	Paramètres rapports
63000–63999	Copropriété	Syndicat, Budgets, Charges

2.2 Nommage des fichiers

Type	Pattern fichier	Exemple
Table	<code>Table [ID] [NomClasse].Table.php</code>	<code>Table 53000 Bail.Table.php</code>
Codeunit	<code>Codeunit [ID] [Nom]Management.php</code>	<code>Codeunit 53000 BailManagement.php</code>
Enum	<code>Enum [ID] [Nom].enum.php</code>	<code>Enum 53000 TypeBail.enum.php</code>
Event	<code>Codeunit [ID] [Nom]Event.php</code>	<code>Codeunit 60010 BailCreeEvent.php</code>
Listener	<code>Codeunit [ID] [Nom]Listener.php</code>	<code>Codeunit 60011 BailCreeListener.php</code>

2.3 Nommage des champs (Tables)

Contexte	Convention	Exemple
Clé primaire métier	PascalCase sans espace	<code>No, Code, Email</code>
Clé étrangère	<code>[Table]_code</code> ou <code>[Table]_no</code>	<code>Bail_no, Unite_code, Tenant_code</code>
Champ montant XOF	<code>Montant_[libelle]</code>	<code>Montant_Loyer, Montant_Depot</code>
Champ date	<code>Date_[libelle]</code>	<code>Date_Debut, Date_Expiration</code>
Champ statut	<code>Statut</code>	<code>Statut</code>
Booléen	<code>Is_[libelle]</code>	<code>Is_Active, Is_Admin, Is_TVA</code>
Champ texte libre	PascalCase	<code>Description, Notes_Internes</code>

2.4 Règles impératives

1. Toute Table DOIT déclarer au moins une clé primaire via `$this->Keys(...)`
2. Tout champ FK DOIT déclarer `tableRelation: new TableCible()`
3. Tout Codeunit DOIT vérifier auth + tenant en première ligne
4. Toute route DOIT appeler `db->commit()` avant l'echo final
5. Toute route DOIT utiliser `getSecureJsonInput()` pour les données POST/PUT
6. Tout montant XOF DOIT être stocké en INTEGER (pas de DECIMAL)
7. Tout rollback en cas d'erreur : `db->rollback()` dans le bloc catch
8. Les Enums REMPLACENT toutes les chaînes de statut magiques dans les Tables
9. Les numéros de séquence DOIVENT passer par `NoSeriesManagement::getNextNo()`
10. Un Codeunit n'accède JAMAIS directement à `$_GET`, `$_POST`, `$_SERVER` – via helpers uniquement

3. STRUCTURE COMPLÈTE DES DOSSIERS

```

Kilie_Immo/
├── index.php                # Point d'entrée unique
├── Routes.php               # Registre centralisé de TOUTES les routes
├── .htaccess                # Réécriture Apache → index.php
├── App/
│   ├── Base system apps/    ← EXISTANT – ne pas modifier
│   │   ├── Tables/
│   │   │   ├── sysTable 9990999 csrfTokenizer.php
│   │   │   ├── sysTable 9999990 AdminUser.Table.php
│   │   │   ├── sysTable 9999991 AdminSession.Table.php
│   │   │   ├── sysTable 9999992 Licence.php
│   │   │   ├── sysTable 9999993 CompanyInfo.php
│   │   │   ├── sysTable 9999994 Authorizations.php
│   │   │   ├── sysTable 9999995 Profile.php
│   │   │   ├── sysTable 9999996 NoSeries.php
│   │   │   ├── sysTable 9999997 Users.Table.php
│   │   │   ├── sysTable 9999998 Session.Table.php
│   │   │   └── sysTable 9999999 Views.Table.php
│   │   ├── Codeunits/
│   │   │   ├── sysCodeunit 9999990 AdminAuthManagement.php
│   │   │   ├── sysCodeunit 9999996 NoSeriesManagement.php
│   │   │   ├── sysCodeunit 9999997 UserManagement.php
│   │   │   └── sysCodeunit 9999998 AuthenticationManagement.php
│   │   └── Enums/
│   │       └── Enum 1 BasicDocumentStatus.enum.php
│   ├── Paramètres/         ← Abonnements SaaS & Config (50000-50999)
│   │   ├── Tables/
│   │   │   ├── Table 50000 Tenant.Table.php
│   │   │   └── Table 50001 PlanAbonnement.Table.php
│   │   ├── Codeunits/
│   │   │   ├── Codeunit 50000 TenantManagement.php
│   │   │   └── Codeunit 50001 AbonnementManagement.php
│   │   └── Enums/
│   │       ├── Enum 50000 StatutTenant.enum.php
│   │       └── Enum 50001 TypePlan.enum.php
│   ├── Gestion immobilière/ ← Biens & Unités (51000-51999)
│   │   ├── Tables/
│   │   │   ├── Table 51000 Bien.Table.php
│   │   │   ├── Table 51001 Unite.Table.php
│   │   │   ├── Table 51002 PhotoBien.Table.php
│   │   │   ├── Table 51003 DocumentFoncier.Table.php
│   │   │   ├── Table 51004 Mandat.Table.php
│   │   │   └── Table 51005 CaracteristiqueBien.Table.php
│   │   ├── Codeunits/
│   │   │   ├── Codeunit 51000 BienManagement.php
│   │   │   ├── Codeunit 51001 UniteManagement.php
│   │   │   └── Codeunit 51002 MandatManagement.php
│   │   └── Enums/
│   │       ├── Enum 51000 TypeBien.enum.php
│   │       ├── Enum 51001 StatutBien.enum.php
│   │       └── Enum 51002 TypeDocumentFoncier.enum.php
│   ├── Parties prenantes/   ← Locataires, Propriétaires, KYC (52000-52999)
│   │   ├── Tables/
│   │   │   ├── Table 52000 Partie.Table.php
│   │   │   ├── Table 52001 PartieIndividu.Table.php
│   │   │   └── Table 52002 PartieEntreprise.Table.php

```

└─ Table 52003 DocumentKYC.Table.php	
└─ Table 52004 Garant.Table.php	
└─ Codeunits/	
└─ Codeunit 52000 PartieManagement.php	
└─ Codeunit 52001 KYCManagement.php	
└─ Enums/	
└─ Enum 52000 TypePartie.enum.php	
└─ Enum 52001 StatutKYC.enum.php	
└─ Enum 52002 TypeRolePartie.enum.php	
└─ Baux & Contrats/	← Baux, Échéanciers, EDL (53000–53999)
└─ Tables/	
└─ Table 53000 Bail.Table.php	
└─ Table 53001 EcheanceBail.Table.php	
└─ Table 53002 ClauseBail.Table.php	
└─ Table 53003 RevisionLoyer.Table.php	
└─ Table 53004 Preavis.Table.php	
└─ Table 53005 EtatDesLieux.Table.php	
└─ Table 53006 LigneEtatDesLieux.Table.php	
└─ Table 53007 SignataireBail.Table.php	
└─ Table 53008 DepotGarantie.Table.php	
└─ Codeunits/	
└─ Codeunit 53000 BailManagement.php	
└─ Codeunit 53001 EcheancierManagement.php	
└─ Codeunit 53002 EtatDesLieuxManagement.php	
└─ Codeunit 53003 RevisionLoyerManagement.php	
└─ Enums/	
└─ Enum 53000 TypeBail.enum.php	
└─ Enum 53001 StatutBail.enum.php	
└─ Enum 53002 StatutEcheance.enum.php	
└─ Paiements & Finance/	← Paiements, MoMo, Quittances (54000–54999)
└─ Tables/	
└─ Table 54000 Paiement.Table.php	
└─ Table 54001 TransactionMobileMoney.Table.php	
└─ Table 54002 Quittance.Table.php	
└─ Table 54003 PenaliteRetard.Table.php	
└─ Table 54004 Relance.Table.php	
└─ Table 54005 ReleveProprietaire.Table.php	
└─ Table 54006 LigneReleveProprietaire.Table.php	
└─ Table 54007 Remboursement.Table.php	
└─ Codeunits/	
└─ Codeunit 54000 PaiementManagement.php	
└─ Codeunit 54001 MobileMoneyManagement.php	
└─ Codeunit 54002 QuittanceManagement.php	
└─ Codeunit 54003 RelanceManagement.php	
└─ Enums/	
└─ Enum 54000 CanalPaiement.enum.php	
└─ Enum 54001 StatutPaiement.enum.php	
└─ Enum 54002 StatutTransactionMM.enum.php	
└─ Comptabilité/	← Plan comptable, Journaux (55000–55999)
└─ Tables/	
└─ Table 55000 PlanComptable.Table.php	
└─ Table 55001 PeriodeFiscale.Table.php	
└─ Table 55002 EcritureComptable.Table.php	
└─ Table 55003 LigneEcritureComptable.Table.php	
└─ Table 55004 DeclarationFiscale.Table.php	
└─ Codeunits/	
└─ Codeunit 55000 ComptabiliteManagement.php	
└─ Codeunit 55001 DeclarationFiscaleManagement.php	
└─ Enums/	

- └─ Enum 55000 TypeCompte.enum.php
 - └─ Enum 55001 TypeEcriture.enum.php
- ─ Maintenance/ ← OI, Prestataires, Devis (56000-56999)
 - └─ Tables/
 - └─ Table 56000 Prestataire.Table.php
 - └─ Table 56001 OrdreTravaux.Table.php
 - └─ Table 56002 DevisOT.Table.php
 - └─ Table 56003 LigneDevisOT.Table.php
 - └─ Table 56004 FactureOT.Table.php
 - └─ Codeunits/
 - └─ Codeunit 56000 MaintenanceManagement.php
 - └─ Codeunit 56001 PrestataireManagement.php
 - └─ Enums/
 - └─ Enum 56000 TypeTravaux.enum.php
 - └─ Enum 56001 StatutOrdreTravaux.enum.php
- ─ CRM/ ← Prospects, Pipeline (57000-57999)
 - └─ Tables/
 - └─ Table 57000 Prospect.Table.php
 - └─ Table 57001 InteretProspect.Table.php
 - └─ Table 57002 ActiviteCRM.Table.php
 - └─ Table 57003 SourceProspect.Table.php
 - └─ Codeunits/
 - └─ Codeunit 57000 CRMManagement.php
 - └─ Enums/
 - └─ Enum 57000 StatutProspect.enum.php
 - └─ Enum 57001 TypeActiviteCRM.enum.php
- ─ Ventes immobilières/ ← Mandats vente, Offres (58000-58999)
 - └─ Tables/
 - └─ Table 58000 MandatVente.Table.php
 - └─ Table 58001 OffreAchat.Table.php
 - └─ Table 58002 CompromisVente.Table.php
 - └─ Codeunits/
 - └─ Codeunit 58000 VenteManagement.php
 - └─ Enums/
 - └─ Enum 58000 StatutOffre.enum.php
- ─ GED/ ← Documents, Stockage (59000-59999)
 - └─ Tables/
 - └─ Table 59000 Document.Table.php
 - └─ Table 59001 DossierDocument.Table.php
 - └─ Table 59002 PartageDocument.Table.php
 - └─ Codeunits/
 - └─ Codeunit 59000 GEDManagement.php
 - └─ Enums/
 - └─ Enum 59000 TypeDocument.enum.php
- ─ Notifications/ ← SMS, Email, Push (60000-60999)
 - └─ Tables/
 - └─ Table 60000 TemplateNotification.Table.php
 - └─ Table 60001 FileNotification.Table.php
 - └─ Table 60002 LogNotification.Table.php
 - └─ Codeunits/
 - └─ Codeunit 60000 NotificationManagement.php
 - └─ Codeunit 60001 SMSAdapter.php
 - └─ Codeunit 60002 EmailAdapter.php
 - └─ Enums/
 - └─ Enum 60000 CanalNotification.enum.php
 - └─ Enum 60001 StatutNotification.enum.php

Intelligence artificielle/	← Solvabilité, Estimation (61000-61999)
Tables/	
Table 61000 ScoreSolvabilite.Table.php	
Table 61001 EstimationBien.Table.php	
Table 61002 ExtractionOCR.Table.php	
Codeunits/	
Codeunit 61000 SolvabiliteManagement.php	
Codeunit 61001 EstimationManagement.php	
Enums/	
Enum 61000 GradeSolvabilite.enum.php	
Reporting/	← Rapports & Exports (62000-62999)
Tables/	
Table 62000 ParametreRapport.Table.php	
Codeunits/	
Codeunit 62000 ReportingManagement.php	
Codeunit 62001 ExportManagement.php	
Copropriété/	← Syndicat, Budgets, AG (63000-63999)
Tables/	
Table 63000 Copropriete.Table.php	
Table 63001 LotCopropriete.Table.php	
Table 63002 BudgetCopropriete.Table.php	
Table 63003 ChargeCopropriete.Table.php	
Table 63004 AssembléeGenerale.Table.php	
Codeunits/	
Codeunit 63000 CoproprieteManagement.php	
Enums/	
Enum 63000 TypeLotCopropriete.enum.php	
vendor/	← Framework core – ne pas modifier
meta/	
Router.php	
global.config.php	
API/App.php	
Database/Database.php	
Objects/Tables/Table.class.php	
Libs/	
FPDF/	
sysCodeunit APIRestCaller.php	
ViewerJS/	
security/Security.class.php	
app.main.php	← Auto-loader Tables + Codeunits
configs/configs.php	← SystemUpdateSchema
env.php	← Config DB
public/	← Frontend SPA (Next.js build)
index.html	
logs/	← Logs applicatifs
docs/	← Documentation projet

4. EXPLICATION DES COMPOSANTS

4.1 TABLE

Rôle : Représente une table MySQL et son ORM. Définit les champs, les clés, les relations et les triggers métier. C'est l'équivalent d'un **Active Record** couplé à la définition du schéma.

Structure systématique :

```

class Bail extends Table {

    public function __construct() {
        parent::__construct(53000, 'bail');    // (ID, nom_table_mysql)

        // Champ 1 : Clé primaire métier
        $this->field(1, 'No',                FieldType::text(20),        caption: 'N° Bail');
        // Champ 2 : FK vers Tenant (isolation multi-tenant – OBLIGATOIRE sur toutes les tables)
        $this->field(2, 'Tenant_code', FieldType::text(50),        caption: 'Tenant');
        // Champ 3 : FK vers Unite
        $this->field(3, 'Unite_code',  FieldType::text(20),        tableRelation: new Unite(),
caption: 'Unité');
        // Champ 4 : FK vers Partie (locataire)
        $this->field(4, 'Locataire_code', FieldType::text(20), tableRelation: new Partie(),
caption: 'Locataire');
        // Champ 5 : Enum via options
        $this->field(5, 'Type_bail',  FieldType::text(50),        options: TypeBail::listValues());
        // Champ 6 : Statut via Enum
        $this->field(6, 'Statut',      FieldType::text(30),        options:
StatutBail::listValues());
        // Champ 7 : Montant XOF – INTEGER (jamais DECIMAL)
        $this->field(7, 'Montant_Loyer_HT', FieldType::integer());
        // ...autres champs

        // Clé primaire (unique composite)
        $this->Keys('No', 'Tenant_code');
    }

    // Trigger avant INSERT
    public function onInsert() {
        // Générer le numéro si absent
        if (IsNullOrEmptyString($this->No->value))
            $this->Validate('No', NoSeriesManagement::getNextNo('BAL'));

        // Calculer le TTC
        if ($this->Is_TVA->value == '1')
            $this->Validate('Montant_Loyer_TTC',
                (int)$this->Montant_Loyer_HT->value
                + (int)round($this->Montant_Loyer_HT->value * 0.18));
        else
            $this->Validate('Montant_Loyer_TTC', $this->Montant_Loyer_HT->value);

        // Statut initial
        if (IsNullOrEmptyString($this->Statut->value))
            $this->Validate('Statut', StatutBail::Brouillon->value);
    }

    // Trigger avant MODIFY
    public function onModify() {
        // Recalculer TTC si loyer modifié
    }

    // Trigger avant DELETE
    public function onDelete() {
        // Vérifier qu'aucun paiement n'est lié avant suppression
        $paiement = new Paiement();
        $paiement->setRange('Bail_no', $this->No->value);
        if ($paiement->FindFirst())
            Error("Impossible de supprimer un bail ayant des paiements enregistrés");
    }
}

```



```

    }
}

```

Champs système automatiques (hérités de `Table.class.php`) :

- `created_at` — DATETIME auto à l'INSERT
- `modified_at` — DATETIME auto à chaque MODIFY
- `deleted_at` — DATETIME pour soft delete
- `Id_slug` — VARCHAR(128) unique (MD5)
- `Id_Incr` — AUTO_INCREMENT

Règle FK : tout champ lié à une autre table DOIT déclarer `tableRelation: new TableCible()`. Le framework vérifie l'intégrité référentielle à chaque INSERT/MODIFY.

4.2 CODEUNIT

Rôle : Classe de logique métier — toutes les méthodes sont statiques. Pas d'état. Orchestre les Tables, applique les règles métier complexes, appelle les services externes. Équivalent d'un **Service Layer**.

Structure systématique :

```

class BailManagement {

    // — CREATE —
    public static function creerBail(array $data): string {
        header('Content-Type: application/json');
        $token = getAuthorizationToken();
        if (!AuthenticationManagement::auth($token)) Unauthorized('Session expirée');
        $tenantCode = getSessionTenant($token);

        // Validation des champs requis
        $rules = [
            'Unite_code'      => ['required' => true],
            'Locataire_code'  => ['required' => true],
            'Date_Debut'      => ['required' => true],
            'Montant_Loyer_HT' => ['required' => true],
        ];
        $v = Security::validateInput($data, $rules);
        if ($v !== true) return json_encode(['status' => 400, 'message' => end($v)]);

        // Vérifier que l'unité est disponible
        $unite = new Unite();
        if (!$unite->get($data['Unite_code'] ?? '')) return json_encode(['status' => 404,
            'message' => 'Unité introuvable']);
        if ($unite->Statut->value !== StatutBien::Disponible->value)
            return json_encode(['status' => 409, 'message' => 'Cette unité est déjà occupée']);

        // Créer le bail
        $bail = new Bail();
        foreach ($data as $k => $v) $bail->Validate($k, Security::sanitizeInput($v, 'string'));
        $bail->Validate('Tenant_code', $tenantCode);
        $result = $bail->Insert();

        // Générer l'échéancier
        EcheancierManagement::genererEcheancier($bail->No->value, $tenantCode);

        // Marquer l'unité comme occupée
        $unite->Validate('Statut', StatutBien::Occupe->value);
        $unite->Modify();
    }
}

```

```

// Déclencher les événements
EventDispatcher::publish('BailCree', [
    'bail_no'      => $bail->No->value,
    'tenant_code'  => $tenantCode,
    'unite_code'   => $bail->Unite_code->value,
    'locataire_code'=> $bail->Locataire_code->value,
]);

db->commit();
return json_encode($result);
}

// — READ —
public static function getBail(string $no): string {
    header('Content-Type: application/json');
    $token = getAthorizationToken();
    if (!AuthenticationManagement::auth($token)) Unauthorized('Session expirée');
    $tenantCode = getSessionTenant($token);

    $bail = new Bail();
    $bail->setRange('Tenant_code', $tenantCode);
    if (!$bail->get($no, $tenantCode)) return json_encode(['status' => 404, 'message' =>
'Bail introuvable']);

    $result = [];
    foreach ($bail->_fields as $n => $f) $result[$n] = $f->value;
    db->commit();
    return json_encode(['status' => 200, 'result' => $result]);
}

// — LIST —
public static function listerBaux(?string $statut = null): string {
    header('Content-Type: application/json');
    $token = getAthorizationToken();
    if (!AuthenticationManagement::auth($token)) Unauthorized('Session expirée');
    $tenantCode = getSessionTenant($token);

    $bail = new Bail();
    $bail->setRange('Tenant_code', $tenantCode);
    if ($statut) $bail->setFilter('Statut', '=%1', $statut);

    $result = [];
    if ($bail->FindSet())
        foreach ($bail->recordSet as $rec) {
            $r = [];
            foreach ($rec->_fields as $n => $f) $r[$n] = $f->value;
            $result[] = $r;
        }
    db->commit();
    return json_encode(['status' => 200, 'result' => $result]);
}

// — UPDATE —
public static function modifierBail(string $no, array $data): string {
    // ...
}

// — ACTIONS MÉTIER —
public static function signerBail(string $no): string { /* ... */ }
public static function activerBail(string $no): string { /* ... */ }
public static function donnerPreavis(string $no, array $data): string { /* ... */ }

```

```

    public static function resilierBail(string $no, array $data): string { /* ... */ }
    public static function reviserLoyer(string $no, array $data): string { /* ... */ }
}

```

4.3 ENUM

Rôle : Définit les valeurs possibles d'un champ statut ou d'une liste fermée. PHP 8.1 backed enums. Remplace toutes les "magic strings" dans le code.

Structure systématique :

```

Enum StatutBail: string {

    case Brouillon           = 'Brouillon';
    case EnAttentSignature   = 'En attente signature';
    case Signé               = 'Signé';
    case Actif               = 'Actif';
    case EnRevision          = 'En révision';
    case Preavis             = 'Préavis';
    case EnSortie            = 'En sortie';
    case Termine             = 'Terminé';
    case Annule              = 'Annulé';
    case Contentieux         = 'Contentieux';

    // Utilitaire – liste pour les options de champ Table
    public static function listValues(): array {
        return array_column(self::cases(), 'value');
    }

    // Logique de transition – quelles transitions sont autorisées
    public function peutPasserA(self $cible): bool {
        return match($this) {
            self::Brouillon           => in_array($cible, [self::EnAttentSignature, self::Annule]),
            self::EnAttentSignature   => in_array($cible, [self::Signé, self::Annule]),
            self::Signé               => in_array($cible, [self::Actif, self::Annule]),
            self::Actif               => in_array($cible, [self::EnRevision, self::Preavis,
self::Contentieux]),
            self::Preavis             => in_array($cible, [self::EnSortie, self::Actif]),
            self::EnSortie            => in_array($cible, [self::Termine]),
            default                   => false,
        };
    }
}

```

4.4 EVENTDISPATCHER (Codeunit transversal)

Rôle : Découple les modules entre eux. Un Codeunit publie un événement après une opération critique. D'autres Codeunits s'y abonnent pour réagir (notifications, comptabilité, mise à jour d'état). Pattern **Observer** adapté au contexte PHP synchrone.

Implémentation : App/Base system apps/Codeunits/sysCodeunit 9990001 EventDispatcher.php

```

class EventDispatcher {

    // Registre des abonnés : ['NomEvenement' => [callable, callable, ...]]
    private static array $_subscribers = [];
}

```

```
// S'abonner à un événement (appelé à l'initialisation, dans vendor/app.main.php)
public static function subscribe(string $eventName, callable $handler): void {
    self::$_subscribers[$eventName][] = $handler;
}

// Publier un événement et déclencher tous ses abonnés
public static function publish(string $eventName, array $payload = []): void {
    foreach (self::$_subscribers[$eventName] ?? [] as $handler) {
        try {
            call_user_func($handler, $payload);
        } catch (Exception $e) {
            // Logger l'erreur sans interrompre le flux principal
            error_log("[EventDispatcher] Erreur listener '$eventName': " . $e->getMessage());
        }
    }
}
}
```

Enregistrement des abonnés dans `vendor/app.main.php` (après le chargement des classes) :

```
// — Abonnements événements —————
EventDispatcher::subscribe('BailCree', [ComptabiliteManagement::class, 'onBailCree']);
EventDispatcher::subscribe('BailCree', [NotificationManagement::class, 'onBailCree']);
EventDispatcher::subscribe('PaieementConfirme', [EcheancierManagement::class,
'onPaieementConfirme']);
EventDispatcher::subscribe('PaieementConfirme', [ComptabiliteManagement::class,
'onPaieementConfirme']);
EventDispatcher::subscribe('PaieementConfirme', [QuittanceManagement::class,
'onPaieementConfirme']);
EventDispatcher::subscribe('EcheanceEnRetard', [RelanceManagement::class,
'onEcheanceEnRetard']);
EventDispatcher::subscribe('EcheanceEnRetard', [PenaliteManagement::class,
'onEcheanceEnRetard']);
EventDispatcher::subscribe('BailResilie', [UniteManagement::class, 'onBailResilie']);
EventDispatcher::subscribe('BailResilie', [FinanceManagement::class, 'onBailResilie']);
EventDispatcher::subscribe('OITermine', [ComptabiliteManagement::class, 'onOITermine']);
EventDispatcher::subscribe('KYCValide', [PartieManagement::class, 'onKYCValide']);
```

4.5 LISTENERS (Méthodes statiques dans les Codeunits)

Rôle : Les listeners ne sont pas des classes séparées — ce sont des méthodes statiques dans les Codeunits existants, préfixées `on[NomEvenement]`. Ils sont enregistrés dans `EventDispatcher::subscribe()`.

Exemple — ComptabiliteManagement listens to PaieementConfirme :

```
class ComptabiliteManagement {

    // ... méthodes principales ...

    // — LISTENERS —————
    public static function onPaieementConfirme(array $payload): void {
        // $payload = ['paieement_no' => ..., 'montant' => ..., 'bail_no' => ...]
        // Passer l'écriture comptable automatique
        self::passerEcritureEncaissement(
            $payload['paieement_no'],
            $payload['montant'],
            $payload['tenant_code']
        );
    }
}
```

```

    );
}

public static function onBailCree(array $payload): void {
    // Créer les comptes analytiques si nécessaire
}

public static function onOITermine(array $payload): void {
    // Passer l'écriture de charge maintenance
}
}

```

Exemple — NotificationManagement listens to BailCree :

```

class NotificationManagement {

    public static function onBailCree(array $payload): void {
        // Envoyer notification de confirmation au locataire et au propriétaire
        self::envoyerNotification(
            $payload['locataire_code'],
            'BAIL_CREE',
            $payload
        );
    }
}

```

4.6 ROUTES — Organisation par module

Les routes sont **toutes centralisées dans Routes.php**, organisées par section avec des délimiteurs visuels. Chaque route appelle directement un Codeunit.

Structure de Routes.php :

```

// =====
// BAUX & CONTRATS
// =====
App::route('GET',    '/baux',                fn() => BailManagement::listerBaux($_GET['statut']
?? null));
App::route('POST',   '/baux',                fn() =>
BailManagement::creerBail(getSecureJsonInput() ?? []));
App::route('GET',    '/baux/{no}',           fn(string $no) =>
BailManagement::getBail(urldecode($no)));
App::route('PUT',    '/baux/{no}',           fn(string $no) =>
BailManagement::modifierBail(urldecode($no), getSecureJsonInput() ?? []));
App::route('PUT',    '/baux/{no}/signer',    fn(string $no) =>
BailManagement::signerBail(urldecode($no)));
App::route('PUT',    '/baux/{no}/activer',   fn(string $no) =>
BailManagement::activerBail(urldecode($no)));
App::route('PUT',    '/baux/{no}/preavis',   fn(string $no) =>
BailManagement::donnerPreavis(urldecode($no), getSecureJsonInput() ?? []));
App::route('PUT',    '/baux/{no}/resilier',  fn(string $no) =>
BailManagement::resilierBail(urldecode($no), getSecureJsonInput() ?? []));
App::route('PUT',    '/baux/{no}/reviser-loyer', fn(string $no) =>
RevisionLoyerManagement::reviser(urldecode($no), getSecureJsonInput() ?? []));
App::route('GET',    '/baux/{no}/echeances', fn(string $no) =>
EcheancierManagement::listerEcheances(urldecode($no)));
App::route('GET',    '/baux/{no}/quittances', fn(string $no) =>

```

```
QuittanceManagement::listerParBail(urldecode($no)));
App::route('POST', '/baux/{no}/etat-des-lieux', fn(string $no) =>
EtatDesLieuxManagement::creer(urldecode($no), getSecureJsonInput() ?? []));
```

5. CATALOGUE COMPLET DES TABLES

Module : Gestion immobilière (51000–51999)

ID	Classe	Table MySQL	Clé(s)	Description
51000	Bien	bien	No, Tenant_code	Bien immobilier
51001	Unite	unite	Code, Tenant_code	Unité d'un bien
51002	PhotoBien	photo_bien	Id_slug	Photos CDN
51003	DocumentFoncier	document_foncier	No, Tenant_code	TF, ACD, ACV
51004	Mandat	mandat	No, Tenant_code	Mandat gestion/location
51005	CaracteristiqueBien	caracteristique_bien	Bien_no, Code	Équipements

Module : Parties prenantes (52000–52999)

ID	Classe	Table MySQL	Clé(s)	Description
52000	Partie	partie	Code, Tenant_code	Locataire ou Propriétaire
52001	PartieIndividu	partie_individu	Partie_code, Tenant_code	Extension personne physique
52002	PartieEntreprise	partie_entreprise	Partie_code, Tenant_code	Extension personne morale
52003	DocumentKYC	document_kyc	Id_slug	Docs KYC uploadés
52004	Garant	garant	Locataire_code, Garant_code	Caution solidaire

Module : Baux & Contrats (53000–53999)

ID	Classe	Table MySQL	Clé(s)	Description
53000	Bail	bail	No, Tenant_code	Contrat de location
53001	EcheanceBail	echeance_bail	Bail_no, Annee, Mois	Ligne échéancier
53002	ClauseBail	clause_bail	Bail_no, No_Ligne	Clauses spéciales
53003	RevisionLoyer	revision_loyer	No, Tenant_code	Historique révisions
53004	Preavis	preavis	No, Tenant_code	Préavis de résiliation
53005	EtatDesLieux	etat_des_lieux	No, Tenant_code	EDL entrée/sortie
53006	LigneEtatDesLieux	ligne_edl	EDL_no, No_Ligne	Détail pièce/élément
53007	SignataireBail	signataire_bail	Bail_no, Partie_code	Suivi signatures
53008	DepotGarantie	depot_garantie	Bail_no	Cycle vie dépôt

Module : Paiements & Finance (54000–54999)

ID	Classe	Table MySQL	Clé(s)	Description
54000	Paieiment	paieiment	No, Tenant_code	Paieiment encaissé

ID	Classe	Table MySQL	Clé(s)	Description
54001	TransactionMobileMoney	transaction_mm	Idempotency_key	Tx CinetPay
54002	Quittance	quittance	No, Tenant_code	Quittance de loyer
54003	PenaliteRetard	penalite_retard	Echeance_no, Tenant_code	Pénalités calculées
54004	Relance	relance	Id_slug	Journal relances
54005	ReleveProprietaire	releve_proprietaire	No, Tenant_code	Relevé mensuel
54006	LigneReleveProprietaire	ligne_releve	Releve_no, No_Ligne	Détail relevé
54007	Remboursement	remboursement	No, Tenant_code	Remboursements

Module : Comptabilité (55000–55999)

ID	Classe	Table MySQL	Clé(s)	Description
55000	PlanComptable	plan_comptable	Code_Compte, Tenant_code	Compte (411, 706...)
55001	PeriodeFiscale	periode_fiscale	Annee, Mois, Tenant_code	Exercice comptable
55002	EcritureComptable	ecriture_comptable	No, Tenant_code	En-tête journal
55003	LigneEcritureComptable	ligne_ecriture	Ecriture_no, No_Ligne	Débit / Crédit
55004	DeclarationFiscale	declaration_fiscale	No, Tenant_code	TVA, Retenue

Autres modules (résumé)

Module	Plage	Tables principales
Maintenance	56000–56999	Prestataire, OrdreTravaux, DevisOT, FactureOT
CRM	57000–57999	Prospect, ActiviteCRM, InteretProspect
Ventes	58000–58999	MandatVente, OffreAchat, CompromisVente
GED	59000–59999	Document, DossierDocument, PartageDocument
Notifications	60000–60999	TemplateNotification, FileNotification, LogNotification
IA	61000–61999	ScoreSolvabilite, EstimationBien, ExtractionOCR
Reporting	62000–62999	ParametreRapport
Copropriété	63000–63999	Copropriete, LotCopropriete, BudgetCopropriete, ChargeCopropriete

6. CATALOGUE COMPLET DES CODEUNITS

ID	Classe	Méthodes principales
50000	TenantManagement	creerTenant, suspendre, activer, upgraderPlan, getLimites
50001	AbonnementManagement	souscrire, renouveler, annuler, getPlansDisponibles
51000	BienManagement	creerBien, getBien, listerBiens, modifierBien, changerStatut, supprimerBien
51001	UniteManagement	creerUnite, getUnite, listerUnites, libererUnite, occuperUnite
51002	MandatManagement	creerMandat, resilierMandat, getMandat, listerMandats
52000	PartieManagement	creerPartie, getPartie, listerParties, changerStatutKYC

ID	Classe	Méthodes principales
52001	KYCManagement	soumettreDocument, validerKYC, rejeterKYC, getStatutKYC
53000	BailManagement	creerBail, getBail, listerBaux, signerBail, activerBail, donnerPreavis, resilierBail
53001	EcheancierManagement	genererEcheancier, listerEcheances, getEcheancesEnRetard, onPaieementConfirme
53002	EtatDesLieuxManagement	creer, ajouterLigne, completer, signer
53003	RevisionLoyerManagement	proposer, approuver, rejeter, appliquer, calculerNouveauLoyer
54000	PaieementManagement	enregistrerPaieement, confirmerPaieement, getPaieement, listerPaieements
54001	MobileMoneyManagement	initierPaieement, webhookCinetPay, getStatutTransaction, rembourser
54002	QuittanceManagement	genererQuittance, getQuittance, listerParBail, envoyerParEmail, onPaieementConfirme
54003	RelanceManagement	relancerEcheancesEnRetard, envoyerRelance, listerRelances, onEcheanceEnRetard
55000	ComptabiliteManagement	passerEcriture, getBalance, getJournal, cloturerPeriode, onPaieementConfirme, onBailCree
55001	DeclarationFiscaleManagement	calculerTVA, calculerRetenue, validerDeclaration, getDeclarations
56000	MaintenanceManagement	creerOI, assignerPrestataire, demarrer, terminer, cloturerOI
56001	PrestataireManagement	creerPrestataire, getPrestataire, listerPrestataires, noterPrestataire
57000	CRMMangement	creerProspect, qualifierProspect, planifierVisite, convertirEnPartie, archiverProspect
58000	VenteManagement	creerMandatVente, soumettreOffre, accepterOffre, signerCompromis
59000	GEDManagement	uploadDocument, getDocument, listerDocuments, genererLienPartage, supprimerDocument
60000	NotificationManagement	envoyerNotification, planifier, getTemplate, onBailCree, onPaieementConfirme
60001	SMSAdapter	envoyer (via Infobip/Orange API)
60002	EmailAdapter	envoyer (via SMTP/SES)
61000	SolvabiliteManagement	calculerScore, getScore, surclasserScore, onKYCValide
61001	EstimationManagement	estimerLoyer, estimerValeur, getEstimation
62000	ReportingManagement	getDashboard, getTauxOccupation, getCA, getImpayeesSynthese
62001	ExportManagement	exporterPDF, exporterCSV, exporterExcel
63000	CoproprieteManagement	creerCopropriete, repartirCharges, appellerCharges, convoquerAG

7. CATALOGUE DES ENUMS

ID	Enum	Valeurs
50000	StatutTenant	Essai, Actif, Suspendu, Expiré, Annulé
50001	TypePlan	Starter, Pro, Enterprise
51000	TypeBien	Villa, Appartement, Studio, Bureau, Local_Commercial, Entrepot, Terrain, Immeuble, Autre
51001	StatutBien	Disponible, Occupe, Reserve, EnVente, Vendu, Suspendu, Archive
51002	TypeDocumentFoncier	TF, ACD, ACP, AV, Permis_Construire, Autre
52000	TypePartie	Proprietaire, Locataire, Garant, Acheteur, Vendeur, Prestataire
52001	StatutKYC	Non_Requis, En_Attente, En_Examen, Valide, Rejete, Expire
53000	TypeBail	Habitation_Non_Meuble, Habitation_Meuble, Commercial_Bureau, Commercial_Local, Autre
53001	StatutBail	Brouillon, En_Attente_Signature, Signe, Actif, En_Revision, Preavis, En_Sortie, Termine, Annule, Contentieux
53002	StatutEcheance	En_Attente, Partiel, Paye, Retard, Annule, Contentieux
54000	CanalPaieement	MTN_MoMo, Orange_Money, Wave, Moov_Money, Virement, Espèces, Cheque, Autre
54001	StatutPaieement	En_Attente, Confirme, Echoue, Annule, Rembourse
54002	StatutTransactionMM	Initie, En_Attente, Succes, Echoue, Expire, Rembourse
55000	TypeCompte	Actif, Passif, Charge, Produit, Capitaux
55001	TypeEcriture	Encaissement, Decaissement, OD, Extourne
56000	TypeTravaux	Plomberie, Electricite, Peinture, Menuiserie, Toiture, Climatisation, Urgence, Divers
56001	StatutOrdreTravaux	Cree, Qualifie, Assigne, En_Cours, Termine, Cloture, Annule
57000	StatutProspect	Nouveau, Contacte, Visite_Planifiee, Visite_Realisee, Dossier_Soumis, Converti, Perdu
60000	CanalNotification	SMS, Email, WhatsApp, Push, In_App
61000	GradeSolvabilite	A, B, C, D, E

8. MATRICE DES ÉVÉNEMENTS INTER-MODULES

Événement publié par	Codeunit publisher	Listeners abonnés	Effet
BailCree	BailManagement::creerBail	UniteManagement, ComptabiliteManagement, NotificationManagement	Marque unité OCCUPEE, crée compte analytique, notifie parties
BailSigne	BailManagement::signerBail	GEDManagement, NotificationManagement	Génère PDF bail signé, notifie confirmation

Événement publié par	Codeunit publisher	Listeners abonnés	Effet
BailResilie	BailManagement::resilierBail	UniteManagement, FinanceManagement, NotificationManagement	Libère unité, génère relevé final, notifie
PaielementConfirme	PaielementManagement::confirmerPaielement	EcheancierManagement, ComptabiliteManagement, QuittanceManagement, NotificationManagement	Met à jour échéance, passe écriture, génère quittance, notifie
EcheanceEnRetard	EcheancierManagement (job cron)	RelanceManagement, PenaliteManagement	Déclenche relances, calcule pénalités
MobileMoneyInitie	MobileMoneyManagement::initierPaielement	NotificationManagement	Envoie USSD/lien de paiement
OITermine	MaintenanceManagement::terminer	ComptabiliteManagement, NotificationManagement	Passe écriture de charge, notifie propriétaire
KYCValide	KYCManagement::validerKYC	SolvabiliteManagement, NotificationManagement	Déclenche calcul score solvabilité, notifie locataire
ProspectConverti	CRMManagement::convertirEnPartie	PartieManagement	Crée la Partie à partir du Prospect
ChargeCoproprieteRepartie	CoproprieteManagement::repartirCharges	EcheancierManagement, NotificationManagement	Crée les appels de fonds dans les échéanciers

9. RÉPONSE API — FORMAT STANDARD

Toutes les réponses suivent le format établi par le framework existant :

```
// Succès
json_encode(['status' => 200, 'message' => 'success', 'result' => $data])
json_encode(['status' => 201, 'message' => 'Created', 'result' => $data])

// Erreurs client
json_encode(['status' => 400, 'message' => 'Champ requis manquant'])
```

```
json_encode(['status' => 401, 'message' => 'Session expirée'])
json_encode(['status' => 402, 'message' => 'Abonnement expiré', 'code' =>
'SUBSCRIPTION_EXPIRED'])
json_encode(['status' => 403, 'message' => 'Accès refusé'])
json_encode(['status' => 404, 'message' => 'Ressource introuvable'])
json_encode(['status' => 409, 'message' => 'Conflit : unité déjà occupée'])
json_encode(['status' => 429, 'message' => 'Trop de tentatives'])

// Erreur serveur
json_encode(['status' => 500, 'message' => 'Erreur interne'])
```

10. CHARGEMENT AUTOMATIQUE (app.main.php)

Le framework charge automatiquement toutes les classes depuis `App/**/Tables/*.php` et `App/**/Codeunits/*.php`. Aucune déclaration d'import nécessaire — toute nouvelle Table ou Codeunit est disponible immédiatement après création dans le bon dossier.

Extension de app.main.php pour les Enums et EventDispatcher :

```
// Charger les Enums (à ajouter dans vendor/app.main.php)
foreach (glob("App/*/Enums/*.enum.php") as $file) require_once $file;
foreach (glob("App/Base system apps/Enums/*.enum.php") as $file) require_once $file;

// Charger EventDispatcher avant les Codeunits
require_once "App/Base system apps/Codeunits/sysCodeunit 9990001 EventDispatcher.php";

// [auto-load Tables et Codeunits existant]

// Enregistrer les abonnements APRÈS le chargement de toutes les classes
EventDispatcher::subscribe('BailCree', [ComptabiliteManagement::class, 'onBailCree']);
// ... (voir section 4.4)
```

Étape suivante : implémentation Table par Table, module par module — dans l'ordre : Paramètres → Gestion immobilière → Parties prenantes → Baux & Contrats → Paiements & Finance